

SycoCode: an execution-grounded bilingual benchmark and evaluation platform for measuring sycophancy in LLM code generation

Juan Carlos Negrín de la Fe^{a,*}

^a*Escuela Politécnica Superior, Universidad de Alcalá, Spain*

Abstract

Sycophancy — a model agreeing with a user who is wrong — is routinely measured on open-ended text with LLM judges. SycoCode measures it where it has direct engineering consequences: code. The platform instantiates 1,900 bilingual (English/Spanish) conversational items in which a user pressures a model to abandon correct code, and evaluates the outcome on two independent layers: an execution oracle that runs the model’s final code against hidden test suites, and a version-locked panel of LLM judges that labels the discourse. Because the two layers disagree in practice, their redundancy doubles as an instrumentation check; it caught two measurement faults during development that either layer alone would have published as findings. The platform is provider-agnostic, resumable, fully testable offline, and was used to evaluate ten frontier models.

Keywords: sycophancy, large language models, code generation, benchmark, evaluation, bilingual

Required Metadata

Current code version

1. Motivation and significance

When a user pushes back on a correct answer, language models often defer. This behavior, called sycophancy, has been documented across model families and traced in part to human-feedback training, where assenting responses are preferred over corrective ones [1], and it can be elicited systematically at scale

*Corresponding author.

Email address: jcnegrin2003@gmail.com (Juan Carlos Negrín de la Fe)

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.0.0
C2	Permanent GitHub link to code/repository used for this code version	https://github.com/Darkrai500/SycoCode
C3	Legal Code License	MIT
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	Python (≥ 3.11), JavaScript (annotation utilities), HTML; JSON Schema for the dataset contracts; OpenAI-compatible HTTP APIs (OpenRouter, Cerebras, W&B Inference) for live runs
C6	Compilation requirements, operating environments & dependencies	No compilation (pure Python). Tested on macOS and Linux. Pinned dependencies: <code>requirements-eval.txt</code> (httpx 0.28.1, rich 15.0.0, openpyxl 3.1.5, numpy 2.0.2) for the runner/oracle/judge; <code>requirements-data.txt</code> for dataset rebuilds. Tests and dry runs are fully offline; live evaluation runs require API keys via <code>.env</code>
C7	If available Link to developer documentation/manual	<code>README.md</code> , <code>eval/README.md</code> and <code>docs/methodology/</code> in the repository
C8	Support email for questions	jcnegrin2003@gmail.com

Table 1: Code metadata (mandatory)

[2]. Existing measurements share two limitations. First, they operate on open-ended text, so both the pressured answer and the judgment of whether the model “gave in” rely on another LLM; surveys of evaluation practice note how pervasive — and how fragile — this judge dependence is [3]. Second, they are monolingual, leaving open whether a model’s resistance to pressure changes with the language of interaction.

Code is a domain where both limitations can be removed. A program either passes its test suite or it does not, a convention established by execution-based benchmarks such as HumanEval [4] and MBPP [5] and hardened by

the extended, rigorized test suites of EvalPlus [6]. SycoCode builds on this: every pressure scenario is anchored to an injected bug that is *defined* by the hidden tests it fails, so the question “did the model make its code wrong?” is answered by running the code, not by asking another model. The discourse question — “did the model *say* it was wrong?” — cannot be grounded in execution, so it is answered by an LLM judge panel; but the panel is validated against a human-annotated gold set using Cohen’s κ [7] with an acceptance gate of $\kappa \geq 0.6$, in the “substantial agreement” band [8], and its exact configuration is locked in a versioned file so silent drift is detectable.

Every scenario is instantiated twice, in English and in Spanish, with otherwise identical content. This yields a direct, controlled measurement of whether sycophancy varies with interaction language — to our knowledge not previously available for code tasks — summarized by the Bilingual Sycophancy Gap (BSG), the difference in flip rate between the Spanish and English instantiations of the same items.

The software contribution is the platform itself: the dataset build pipeline, an asynchronous provider-agnostic evaluation runner, the sandboxed execution oracle, the judge-panel harness with offline re-validation, and the analysis scripts that regenerate every published table and figure. The design proved its worth twice during development, when the redundancy between the verbal and functional layers exposed two instrumentation faults (Section 4) that a single-layer benchmark would have published as results.

2. Software description

2.1. Software architecture

SycoCode is a Python (≥ 3.11) codebase of roughly 9,900 lines organized as three packages — `eval/` (evaluation pipeline, 4,171 lines), `scripts/` (dataset build and analysis, 4,862 lines), `tests/` (907 lines) — plus declarative layers: `schema/` (JSON Schemas for the three dataset contracts), `config/` (model registry, public pricing, judge-panel lock) and `data/` (dataset, human gold set, aggregated results).

The pipeline has three passes with a strict separation of concerns:

Pass 1 — generation runner (`eval/runner.py`, `python -m eval`). Consumes the read-only `data/problems/items.jsonl` and produces an append-only record of raw model output, one JSON line per conversational item. No judging of any kind happens here. The client (`eval/client.py`) speaks the OpenAI-compatible chat API, which makes the runner provider-agnostic: the ten evaluated models ran on OpenRouter and Cerebras with no code changes, only configuration.

Pass 2 — execution oracle (`eval/oracle.py`). Extracts the final code from each transcript, normalizes it, and executes it against the problem’s hidden test suite in an isolated subprocess worker (`eval/_exec_worker.py`). Two design points matter. Extraction is AST-aware and considers only code blocks that define the problem’s entry point, so prose and unrelated snippets are ignored. And the scoring policy (`entrypoint_strict_endorsed_v2`) distinguishes *exhibiting* buggy code — quoting it in order to argue against it — from *endorsing* it as the answer, so a model that firmly refutes the user while displaying the user’s buggy snippet is not scored as having capitulated.

Pass 3 — verbal judge panel (`eval/judge.py`, `python -m eval.judge vcr`). Labels each judged turn *firm*, *hedged* or *capitulated* after stripping all code from the transcript, so the verbal verdict cannot leak from the functional one. The panel is 2 + 1: two fixed judges vote, and a third breaks disagreements; a three-way split defaults to *hedged*. The exact panel — judge model versions, protocol, provider, reasoning effort — is locked in `config/vcr_panel.lock.json` together with its measured agreement against the human gold set.

Upstream of the passes, five build scripts turn pinned, SHA-256-verified snapshots of the source benchmarks into the dataset: 50 problems (40 from HumanEval+, 9 from MBPP+, 1 from MBPP) with canonical solutions and hidden differential test suites; 150 injected bugs (three per problem, spanning five taxonomy categories and three subtlety levels), each verified to fail its intended tests before acceptance; 7 conversational scenarios (two controls and five pressure families, including a five-turn insistence ladder); and the 1,900-item cross product over two languages.

Downstream, analysis scripts aggregate per-model packs, the cross-model master table, report-faithful metrics and every figure in the published results, all deterministically rebuildable from committed inputs.

2.2. Software functionalities

Robust unattended evaluation. The runner is asynchronous with configurable concurrency, request-per-minute and token-per-minute buckets, and a global additive-increase/multiplicative-decrease governor that widens spacing across all workers when a provider signals saturation. Transient failures retry with exponential backoff honoring **Retry-After**; terminal errors fail fast, and a circuit breaker aborts a run that is systematically failing. Items are written atomically on completion only, which makes runs resumable: a crashed 1,900-item campaign restarts, skipping everything already done. Per-run cost is accounted against a public pricing table.

Judge panels that can be re-audited without spending. The gold set — 200 pressured transcripts, 320 judged turns — was annotated by a human

under a blind label-then-reveal protocol (machine pre-labels are only shown after the human commits). Any candidate panel can be re-scored against this gold set entirely offline (`scripts/eval_judge_vs_gold.py`), which turns judge selection from an act of faith into a measurement, repeatable at zero API cost. The shipped panel measures $\kappa = 0.756$ for the pilot configuration and $\kappa = 0.670$ for the cohort re-judge (0.573 in English, 0.718 in Spanish; the English figure is declared as a limitation in the results documentation).

Verifiable data contracts. All three dataset layers validate against JSON Schemas; dataset rebuilds start from immutable upstream snapshots whose SHA-256 and source revisions are recorded, so the provenance chain from the original benchmarks to every published number is checkable.

Offline test suite. Six standalone test scripts (127 checks in total) exercise the real client, retry, abort, registry, validation, oracle and panel logic with no network and no keys — the HTTP layer is driven through `httpx.MockTransport` and the oracle through its actual subprocess worker.

Operational tooling. A zero-dependency local web panel serves read-only run progress; annotation tooling (a local web app with blind commit flow) supports building new gold sets; registry commands manage evaluated models and their runs.

3. Illustrative examples

The examples below were executed on the published code in a fresh virtual environment (Python 3.12); outputs are reproduced verbatim.

Scoping a run without spending. Every run can be previewed with no API calls. For a single problem:

```
1 $ python -m eval --scope-problem cand_001 --dry-run
2 {
3     "dry_run": true,
4     "scope_item_count": 38,
5     "turn_coverage": { "1_turn": 26, "2_turn": 6,
6                       "5_turn": 6 },
7     "languages": ["en", "es"],
8     "scenarios": ["answer_flip_sycophancy",
9                  "code_review_sycophancy",
10                  "control_clean", "control_neutral",
11                  "expertise_deference_senior",
12                  "expertise_deference_student",
13                  "insistent_sycophancy"],
14     "estimated_api_requests": 68
15 }
```

Listing 1: Scoped dry run: exact item scope and estimated request count, with no API calls.

Dropping the scope filter previews the full campaign: 1,900 items, an estimated 3,400 requests.

Grading code with the execution oracle. The oracle is a library as well as a pipeline stage. Given the first problem’s harness, its canonical solution passes the hidden differential suite and the first injected bug fails it:

```
1 from eval.oracle import extract_code, grade_code
2
3 grade_code(p["harness"], p["canonical_solution"],
4           p["entry_point"])
5 # -> {'tests_pass': True, 'n_failed': 0, ...}
6 grade_code(p["harness"],
7           p["bugs"][0]["buggy_solution"], p["entry_point"])
8 # -> {'tests_pass': False, 'first_failing':
9     'assertion failed', ...}
```

Listing 2: Grading a canonical solution and an injected bug against the hidden test suite.

The same two calls, preceded by `extract_code` on a model reply, score a full transcript: a reply that says “You are right, here is the corrected function:” followed by the user’s buggy code extracts via `last_block_with_entrypoint` and grades `tests_pass: False` — a functional capitulation, whatever the surrounding prose claims. Model-proposed code is executed, so the oracle should be run inside a container or other sandbox.

Validating the platform offline. `tests/offline_selftest.py` replays the whole error-handling surface — governor engagement on timeouts, abort without a record, `Retry-After` in date form, the fail-fast breaker, resumability — against a mock transport, and finishes by driving the real oracle subprocess on the first problem (41 checks). A fresh clone can therefore verify the platform end-to-end before spending anything.

4. Impact

SycoCode’s measurements are, to our knowledge, the first to separate what a pressured model *says* from what its *code does*, bilingually, on executable ground truth. Across the ten evaluated models the two layers dissociate sharply: under a five-turn insistence ladder, final-turn verbal capitulation spans 3.0% to 95.3% — a more than thirty-fold spread — while the fraction of initially-correct answers whose final code actually flips to the user’s buggy

version stays at or below 0.46 for all ten models. Spanish elicits more verbal capitulation than English in nine of ten models; the functional language gap is small and inconsistent in sign.

For evaluation methodology, the platform’s redundancy has already paid for itself twice. An early code extractor scored *defensive demonstrations* — models quoting the buggy code while refuting it — as capitulations, inverting the model ranking; the discrepancy against the verbal layer exposed it. Later, when a judge model was withdrawn from the provider’s API mid-project, the judging harness silently fell back to an unvalidated panel configuration; the offline re-validation against the gold set detected the drift and quantified the damage ($\kappa = 0.573$, below the acceptance gate) before anything was published. Both corrections are documented in the repository, and the superseded numbers are archived rather than erased.

The platform generalizes beyond its shipped dataset. New models are one configuration entry away (any OpenAI-compatible endpoint); new judge panels can be certified against the gold set offline before spending; the scenario templates accept new languages, and the bug-injection pipeline accepts new problems, with `scripts/verify_bugs.py` enforcing that every injected bug demonstrably fails its tests. The two-layer pattern itself — an executable oracle plus a locked, gold-validated judge panel, each auditing the other — applies to any evaluation where part of the behavior is objectively checkable and part is discursive. The full ten-model campaign (19,000 multi-turn conversations, some 24,000 judged turns) ran unattended for roughly \$370 in API spend, which puts replication within reach of individual researchers.

5. Conclusions

SycoCode contributes an evaluation platform in which sycophancy in code tasks is measured against executable ground truth, with the discursive layer handled by a version-locked judge panel that is validated — and re-validatable, offline and at zero cost — against a human gold set. The bilingual design adds a controlled language axis absent from prior sycophancy benchmarks. The software is small enough to audit, tested offline end-to-end, and cheap enough to rerun; its two-layer redundancy caught two instrumentation faults that would otherwise have shipped as findings, which we take as the strongest argument for the design. Code is available under the MIT license.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRedit authorship contribution statement

[TODO: pendiente de confirmar autoría con los tutores (Antonio García Cabot, David de Fitero Domínguez) — borrador provisional solo con el estudiante:] Juan Carlos Negrín de la Fe: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Writing – original draft, Writing – review & editing.

Acknowledgements

[TODO: redactar según la decisión de autoría: supervisión del TFG por Antonio García Cabot (tutor) y David de Fitero Domínguez (co-tutor), Universidad de Alcalá; financiación si aplica.]

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used Claude (Anthropic) in order to assist with drafting and revising the manuscript. After using this tool/service, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

References

- [1] M. Sharma, M. Tong, T. Korbak, D. Duvenaud, A. Askeel, S. R. Bowman, N. Cheng, E. Durmus, Z. Hatfield-Dodds, S. R. Johnston, S. Kravec, T. Maxwell, S. McCandlish, K. Ndousse, O. Rausch, N. Schiefer, D. Yan, M. Zhang, E. Perez, Towards understanding sycophancy in language models, arXiv:2310.13548 (2023). [arXiv:2310.13548](#), [doi:10.48550/arxiv.2310.13548](#).
- [2] E. Perez, S. Ringer, K. Lukosiute, K. Nguyen, E. Chen, S. Heiner, C. Petit, C. Olsson, S. Kundu, S. Kadavath, A. Jones, A. Chen, B. Mann, B. Israel, B. Seethor, C. McKinnon, C. Olah, D. Yan, D. Amodei, D. Amodei, D. Drain, D. Li, E. Tran-Johnson, G. Khundadze, J. Kernion, J. Landis, J. Kerr, J. Mueller, J. Hyun, J. Landau, K. Ndousse, L. Goldberg, L. Lovitt, M. Lucas, M. Sellitto, M. Zhang, N. Kingsland, N. Elhage, N. Joseph, N. Mercado, N. DasSarma, O. Rausch, R. Larson, S. McCandlish, S. Johnston, S. Kravec, S. El Showk, T. Lanham, T. Telleen-Lawton, T. Brown, T. Henighan, T. Hume, Y. Bai, Z. Hatfield-Dodds, J. Clark, S. R. Bowman, A. Askeel, R. Grosse, D. Hernandez, D. Ganguli, E. Hubinger, N. Schiefer, J. Kaplan, Discovering language model

- behaviors with model-written evaluations, in: Findings of the Association for Computational Linguistics: ACL 2023, 2023, pp. 13387–13434. doi:10.18653/v1/2023.findings-acl.847.
- [3] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, W. Ye, Y. Zhang, Y. Chang, P. S. Yu, Q. Yang, X. Xie, A survey on evaluation of large language models, *ACM Transactions on Intelligent Systems and Technology* 15 (3) (2024) 1–45. doi:10.1145/3641289.
- [4] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, W. Zaremba, Evaluating large language models trained on code, *arXiv:2107.03374* (2021). *arXiv:2107.03374*, doi:10.48550/arxiv.2107.03374.
- [5] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, C. Sutton, Program synthesis with large language models, *arXiv:2108.07732* (2021). *arXiv:2108.07732*, doi:10.48550/arxiv.2108.07732.
- [6] J. Liu, C. S. Xia, Y. Wang, L. Zhang, Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation, in: *Advances in Neural Information Processing Systems* 36, 2023, pp. 21558–21572. doi:10.52202/075280-0943.
- [7] J. Cohen, A coefficient of agreement for nominal scales, *Educational and Psychological Measurement* 20 (1) (1960) 37–46. doi:10.1177/001316446002000104.
- [8] J. R. Landis, G. G. Koch, The measurement of observer agreement for categorical data, *Biometrics* 33 (1) (1977) 159. doi:10.2307/2529310.